NeuroDynamics®

General onboarding
NRO-PRO-014-1

DEPARTAMENTO DE PESQUISA E DESENVOLVIMENTO

# Projects practical technologies introduction

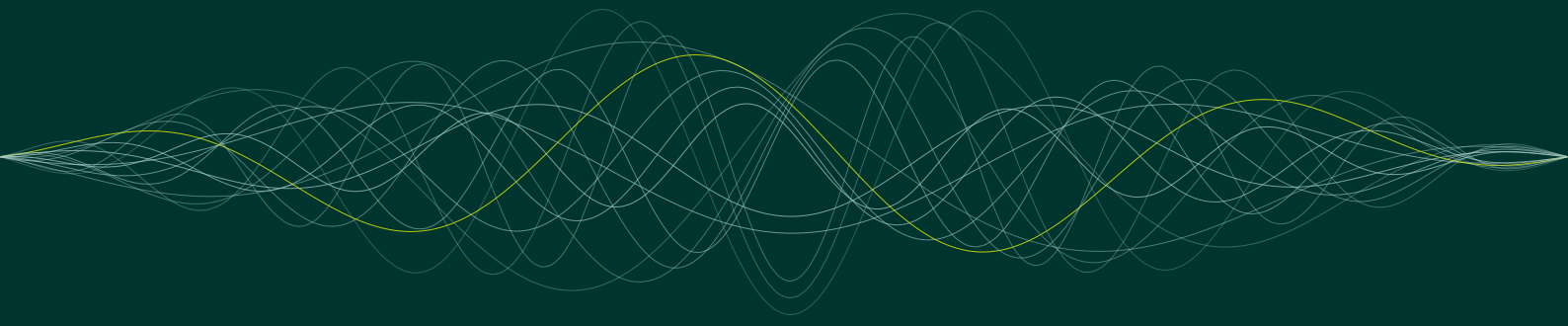
Hardware, firmware e software para dispositivos médicos e sistemas embarcados, do básico ao avançado.


Autoria
Gabriel Matos Veneziano



Data
25 de outubro de 2026

Revisão
A

CONTROLE

# Sobre este documento

| | |
|---|---|
| Objetivo | Conduzir qualquer membro da equipe, do primeiro dia até o nível de quem projeta e revisa sistemas completos, em quatro áreas: hardware, firmware, software e integração. |
| Escopo | Conceitos gerais de sistemas embarcados e dispositivos médicos, cada um localizado no Connect, a plataforma de eletroestimulação da equipe, usada como estudo de caso. |
| Público | Quem está chegando (como curso) e quem já é da equipe (como referência). |
| Pré-requisito | A Parte 0 é obrigatória para todos e precede as trilhas de área. |
| Resultado esperado | Seguir um comando por todas as camadas e localizar onde um problema nasce; implementar uma funcionalidade ponta a ponta; documentar, analisar riscos e validar um sistema completo. |
| Referências normativas | IEC 60601-1, IEC 60601-2-10, ISO 14971 e IEC 62304. |

## Controle do documento

| | |
|---|---|
| Código | NRO-PRO-014-1 |
| Revisão | A |
| Departamento emissor | Departamento de Pesquisa e Desenvolvimento |
| Tipo | General onboarding |
| Data | 25 de outubro de 2026 |
| Redator | Gabriel Matos Veneziano |
| Revisor | a preencher |
| Aprovador | a preencher |

## Histórico de revisões

| DATA | REV. | RESPONSÁVEL | MUDANÇAS |
|---|---|---|---|
| 25/10/2026 | A | Gabriel Matos Veneziano | Primeira edição: 5 partes, 85 tópicos, glossário e ficha de progresso. |

CONTEÚDO

# Sumário

**Boas-vindas** 5

**Como usar este documento** 6

**Mapa das trilhas** 8

**O Connect em uma página** 10

**Parte 0: Fundamentos comuns** 12
0.1 Dispositivos de neuroestimulação . . . 13
0.2 Fisiologia básica para engenharia . . . 13
0.3 Segurança elétrica e normas de equipamento médico . . . 14
0.4 Sistemas numéricos e representação de dados . . . 14
0.5 Linha de comando Linux . . . 15
0.6 Git e fluxo de trabalho . . . 15
0.7 Leitura de documentação técnica . . . 16

**Parte 1: Hardware** 17
1.1 Circuitos elétricos . . . 18
1.2 Componentes passivos . . . 18
1.3 Semicondutores . . . 19
1.4 Instrumentação de bancada . . . 19
1.5 Leitura de esquemático . . . 20
1.6 Amplificadores operacionais . . . 21
1.7 Fontes de corrente controladas por tensão . . . 21
1.8 Medição de corrente . . . 22
1.9 Lógica digital . . . 22
1.10 Conversores DAC e ADC . . . 22
1.11 Chaves analógicas e multiplexação . . . 23
1.12 Relés de estado sólido e isolação . . . 23
1.13 Fontes chaveadas . . . 24
1.14 Gate drivers e ponte H . . . 24
1.15 Gerenciamento de bateria . . . 25
1.16 Sequenciamento de energia e load switches . . . 25
1.17 Proteções de hardware . . . 25
1.18 Integridade de sinal e layout . . . 26
1.19 Validação de hardware . . . 26

**Parte 2: Firmware** 27
2.1 Linguagem C . . . 28
2.2 C++ para embarcados . . . 28
2.3 Arquitetura de microcontroladores . . . 29
2.4 GPIO . . . 29
2.5 Toolchain . . . 29
2.6 Build system . . . 30
2.7 Cortex-M0+ . . . 31
2.8 Startup e linker script . . . 31
2.9 Periféricos de um microcontrolador . . . 32
2.10 Geradores de código de fabricante . . . 32
2.11 Protocolos seriais . . . 32
2.12 Drivers de CIs externos . . . 33
2.13 Gravação e depuração . . . 33

2.14 Qualidade de código . . . 34
2.15 Arquitetura superloop cooperativa . . . 35
2.16 Máquinas de estados . . . 35
2.17 Temporização precisa . . . 36
2.18 Concorrência com interrupções . . . 36
2.19 Tratamento de erros . . . 36
2.20 Persistência . . . 37
2.21 Estado central e sincronização . . . 37
2.22 Protocolo binário próprio . . . 38
2.23 ESP32 e ESP-IDF . . . 38
2.24 FreeRTOS . . . 38
2.25 Bluetooth Low Energy: lado periférico . . . 39
2.26 Ponte entre BLE e UART . . . 39
2.27 Código compartilhado entre plataformas . . . 40
2.28 Firmware de segurança crítica . . . 40
2.29 Testes de firmware . . . 40

**Parte 3: Software** 42
3.1 Linguagem Dart . . . 43
3.2 Fundamentos de Flutter . . . 43
3.3 Ambiente de desenvolvimento . . . 44
3.4 Projetos Android e iOS . . . 44
3.5 Gerência de estado com BLoC . . . 45
3.6 Modularização e injeção de dependência . . . 45
3.7 Arquitetura em camadas . . . 46
3.8 Pacotes locais e monorepo . . . 46
3.9 Serialização e geração de código . . . 46
3.10 Internacionalização . . . 47
3.11 Persistência local . . . 47
3.12 Formulários e validação . . . 48
3.13 Lint e qualidade . . . 48
3.14 Bluetooth Low Energy: lado central . . . 49
3.15 Conversão de bytes . . . 49
3.16 Contrato entre app e firmware . . . 50
3.17 Concorrência no app . . . 50
3.18 Comunicação com backend . . . 50
3.19 Firebase . . . 51
3.20 Tratamento de erros padronizado . . . 51
3.21 Fluxos clínicos no app . . . 52
3.22 Gráficos e visualização . . . 52
3.23 Testes . . . 52
3.24 Distribuição . . . 53

**Parte 4: Integração do sistema** 54
4.1 O caminho completo de um comando . . . 55
4.2 Uma funcionalidade ponta a ponta . . . 55
4.3 Depuração entre camadas . . . 55
4.4 Documentação de engenharia . . . 56
4.5 Análise de segurança do sistema . . . 56
4.6 Verificação e validação . . . 57

**Glossário** 58

**Ficha de progresso** 60

**Como contribuir** 66

INTRODUÇÃO

# Boas-vindas

Este documento é o caminho de formação técnica da equipe NeuroDynamics. Ele leva qualquer membro, do primeiro dia até o nível de quem projeta e revisa sistemas completos, por quatro áreas: hardware, firmware, software e integração.

Os conceitos são gerais e valem para qualquer sistema embarcado ou dispositivo médico. Para que o estudo não fique no abstrato, cada tópico mostra onde aquele conceito aparece no **Connect**, a plataforma de eletroestimulação que a equipe desenvolve. Assim, quem estuda aprende a teoria e, ao mesmo tempo, conhece o produto por dentro.

O documento atende a dois públicos. Para quem está chegando, é um curso: comece pela Parte 0 e siga a trilha da sua área. Para quem já é da equipe, é uma referência: use o sumário para ir direto ao tópico e às fontes de estudo.

| 5 | 85 | 3 | 173 |
|---|---|---|---|
| PARTES | TÓPICOS | NÍVEIS POR TRILHA | EXERCÍCIOS |

## Princípios

**Geral, com os pés no produto**
Cada conceito é ensinado de forma geral e logo depois localizado no Connect.

**Do porquê ao como**
Antes de implementar, entenda por que a solução existe e que problema ela resolve.

**Aprender fazendo**
Todo tópico termina em exercícios práticos, de preferência no código e no esquemático reais.

**Documento vivo**
O documento melhora a cada atualização. Quem estuda um tópico deixa-o melhor para o próximo.

GUIA

# Como usar este documento

O documento tem cinco partes. A Parte 0 é obrigatória para todos. As Partes 1, 2 e 3 são trilhas de área, cada uma com três níveis: básico, intermediário e avançado. A Parte 4 fecha a formação e é comum a todas as áreas.

Também é altamente recomendável que, ao estudar um tópico, você crie o hábito de pesquisar o assunto na internet e de estudar por conta própria, orientado pelas suas necessidades. Guarde as fontes estudadas e contribua com o documento! Nem todos temos a mesma bagagem de conhecimento: o documento serve como guia e exposição dos tópicos mais importantes, mas o verdadeiro aprendizado vem da curiosidade, da descoberta própria e da resolução de problemas. Seja consciente e sincero consigo mesmo ao avançar nos tópicos.

| PARTE | CONTEÚDO | PARA QUEM | TÓPICOS |
|---|---|---|---|
| 0 | **Fundamentos comuns** | Todos, no início | **0.1** a **0.7** (7) |
| 1 | **Hardware** | Trilha de hardware | **1.1** a **1.19** (19) |
| 2 | **Firmware** | Trilha de firmware | **2.1** a **2.29** (29) |
| 3 | **Software** | Trilha de software | **3.1** a **3.24** (24) |
| 4 | **Integração do sistema** | Todos, ao final | **4.1** a **4.6** (6) |

## Anatomia de um tópico

Todos os tópicos seguem o mesmo formato, para que você saiba onde encontrar cada coisa:

| | |
|---|---|
| **1.7 Título** | Número e nome do tópico. O número diz a parte e a ordem sugerida. |
| **Texto de abertura** | O conceito explicado de forma geral, válido para qualquer projeto. |
| NO CONNECT | Quadro que mostra onde o conceito aparece nos repositórios e esquemáticos da equipe. |
| PARA ESTUDAR | Sites, livros, vídeos e documentos internos. Os itens sublinhados são links. |
| PRATIQUE | Exercícios para fixar, quase sempre ligados ao código ou ao hardware real. |

## Como estudar bem

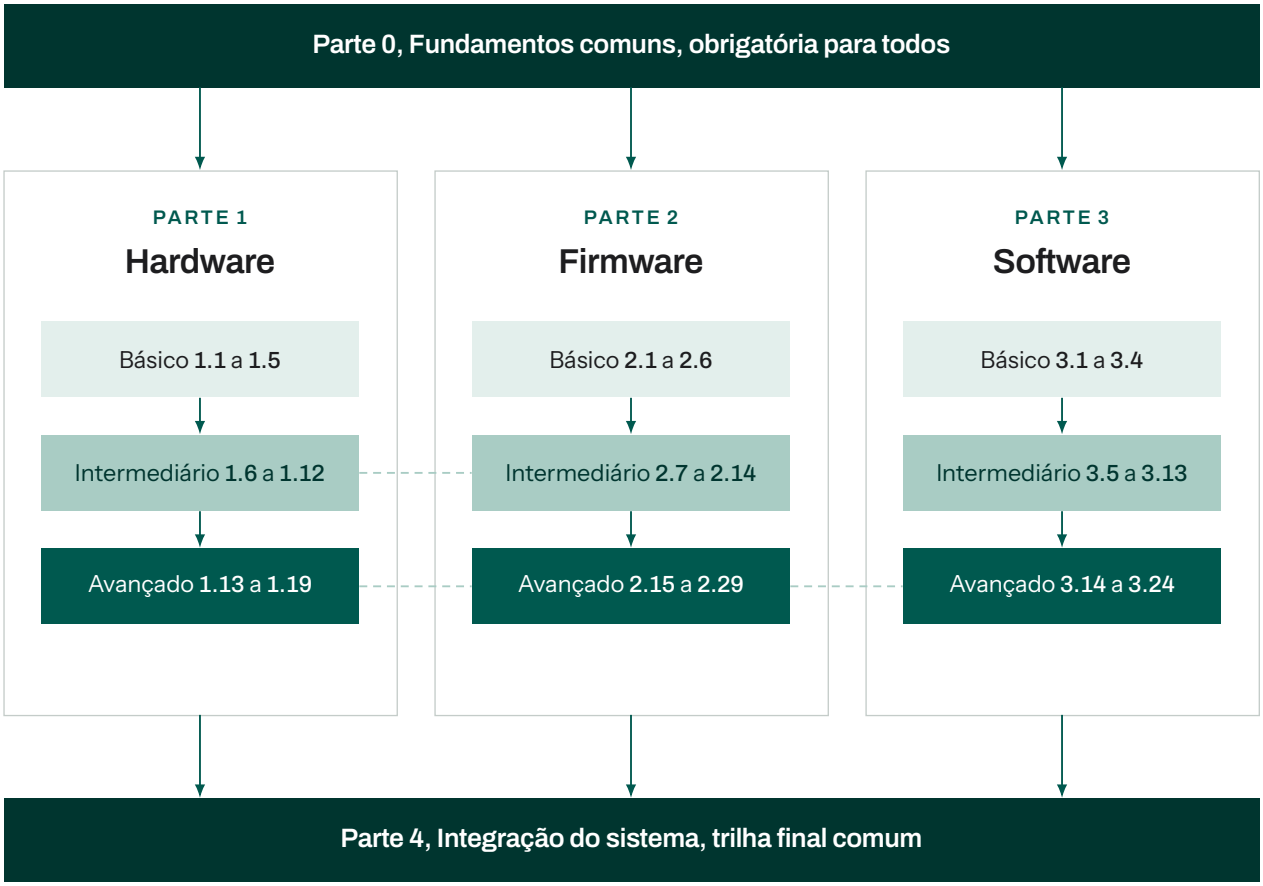
1. Leia o texto de abertura e explique com suas palavras por que aquilo existe.
2. Use as referências para responder aos exercícios. Não é preciso ler tudo de ponta a ponta.
3. Tente os exercícios antes de abrir o código ou pedir ajuda; anote onde travou.

4. Leve o tópico para o repositório: encontre no código ou no esquemático o que o quadro No Connect descreve.
5. Marque o tópico na Ficha de progresso, no fim do documento.
6. E principalmente, anote suas dúvidas e converse com seus colegas de equipe sobre elas. Discuta soluções, resolva problemas e não deixe suas incertezas quanto aos tópicos acumularem.

ROTEIRO

# Mapa das trilhas

Todo membro faz a Parte 0, segue de básico a avançado na trilha da sua área e termina na Parte 4. As linhas tracejadas são pontes: tópicos de trilhas diferentes que tratam do mesmo assunto, cada um do seu lado.




## Pontes entre trilhas

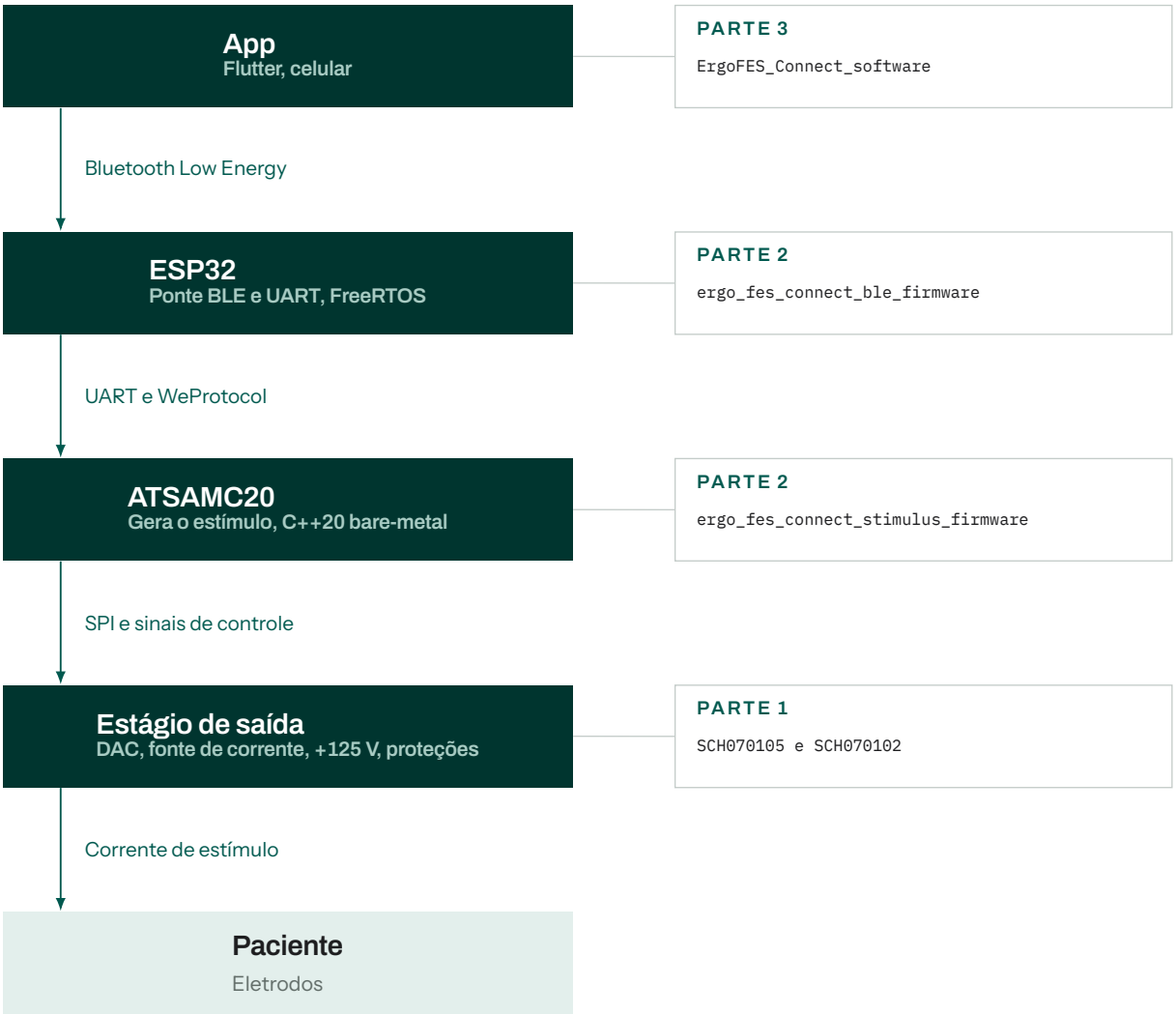
Para atravessar para outra área, estude os tópicos da mesma linha juntos:

| HARDWARE | FIRMWARE | SOFTWARE |
|---|---|---|
| 1.7 a 1.12, fonte de corrente, sense, lógica, DAC, chaves e relés | 2.9 a 2.12, periféricos, protocolos seriais e drivers | — |
| 1.17, proteções de hardware | 2.28, firmware de segurança crítica | — |
| — | 2.25, BLE periférico | 3.14 a 3.16, BLE central, bytes e contrato |
| — | 2.22, protocolo binário | 3.15, conversão de bytes |

ESTUDO DE CASO

# O Connect em uma página

O Connect é a plataforma de eletroestimulação da equipe e o laboratório deste documento. Ele tem os blocos de qualquer dispositivo médico conectado: um aplicativo, um enlace sem fio, processadores com firmware e um estágio de potência que chega ao paciente. Cada bloco vive em um repositório e é estudado em uma trilha.



## Repositórios e documentos de referência

| ÁREA | REFERÊNCIA | TECNOLOGIA | TRILHA |
|---|---|---|---|
| **Firmware stimulus** | `ergo_fes_connect_stimulus_firmware` | ATSAMC20N17A, C++20 bare-metal, MPLAB Harmony | Parte 2 |
| **Firmware BLE** | `ergo_fes_connect_ble_firmware` | ESP32, ESP-IDF, Bluedroid, FreeRTOS | Parte 2 |
| **Aplicativo** | `ErgoFES_Connect_software` (pasta com os 5 repositórios de software) | Flutter (`app_main`), pacotes `bluetooth`, `byte_conversion`, `date_time_helper`, `exception` | Parte 3 |
| **Hardware** | **SCH070105** (CFCU) e **SCH070102** (CTCE) | Esquemáticos das placas | Parte 1 |

PARTE 00

# Fundamentos comuns

A base que todo membro precisa, de qualquer área: como funciona um dispositivo de neuroestimulação, o mínimo de fisiologia e de segurança elétrica, e as ferramentas do dia a dia de engenharia.

## VOCÊ VAI APRENDER A

- √ Explicar a arquitetura de um eletroestimulador e o papel de cada camada.
- √ Trabalhar com binário, hexadecimal, terminal e Git sem travar.
- √ Ler datasheets, esquemáticos e normas com objetivo claro.

## TÓPICOS DESTA PARTE


**0.1** Dispositivos de neuroestimulação

**0.2** Fisiologia básica para engenharia

**0.3** Segurança elétrica e normas de equipamento médico

**0.4** Sistemas numéricos e representação de dados

**0.5** Linha de comando Linux

**0.6** Git e fluxo de trabalho

**0.7** Leitura de documentação técnica

## 0.1 Dispositivos de neuroestimulação

Eletroestimuladores aplicam corrente controlada ao corpo para ativar nervos e músculos (FES, TENS) ou para modular a atividade do cérebro e do sistema vestibular (tDCS, GVS). Por trás de qualquer um deles há a mesma arquitetura: uma interface com o usuário, um enlace de comunicação, um microcontrolador que gera a forma de onda e um estágio de saída que entrega a corrente com segurança.

> **NO CONNECT**
> O Connect segue essa cadeia, nos dois sentidos: app, BLE, ESP32, UART, ATSAM, estágio de saída e paciente. O app vive em `ErgoFES_Connect_software`, a ponte BLE em `ergo_fes_connect_ble_firmware`, a geração do estímulo em `ergo_fes_connect_stimulus_firmware` e o estágio de saída nos esquemáticos `SCH070105` (CFCU) e `SCH070102` (CTCE).

**PARA ESTUDAR**

- Wikipedia: Functional electrical stimulation
- Wikipedia: Transcranial direct-current stimulation
- Wikipedia: Galvanic vestibular stimulation
- Documento técnico interno do Connect e READMEs dos repositórios

**PRATIQUE**

0.1.1 Desenhe o diagrama de blocos de um eletroestimulador genérico e, ao lado, o do Connect. Indique em cada seta o protocolo ou sinal usado.
0.1.2 Para cada repositório do Connect, escreva uma frase dizendo o que ele faz e com quem conversa.
0.1.3 Explique com suas palavras a diferença entre um pulso de FES e a corrente contínua de tDCS.

## 0.2 Fisiologia básica para engenharia

Nervo e músculo, potencial de ação, contração evocada por estímulo elétrico, a interface pele-eletrodo (e sua impedância) e o sistema vestibular. É o mínimo de biologia para entender por que os parâmetros de estímulo têm os limites que têm.

> **NO CONNECT**
> Esses conceitos explicam os parâmetros que o firmware permite configurar, como amplitude e largura de pulso, e por que o hardware precisa detectar eletrodo solto.

**PARA ESTUDAR**

- Neuroscience Online (UTHealth), capítulos de neurônio e músculo
- Livro: Princípios de Neurociências (Kandel et al.), capítulos introdutórios

**PRATIQUE**

0.2.1 Explique por que um pulso precisa de amplitude e duração mínimas para recrutar uma fibra (curva intensidade-duração).
0.2.2 Modele pele e eletrodo como um circuito RC simples e esboce a tensão sobre ele sob um degrau de corrente.

## 0.3 Segurança elétrica e normas de equipamento médico

Efeitos da corrente no corpo, as normas IEC 60601-1 (segurança geral de equipamento médico) e IEC 60601-2-10 (estimuladores de nervo e músculo), a gestão de risco pela ISO 14971 e o ciclo de vida de software médico pela IEC 62304.

> **NO CONNECT**
> O estágio de saída trabalha com +125 V. Por isso o Connect combina proteções em hardware (sobrecorrente, parada de emergência, eletrodo solto) com regras de firmware, como só habilitar a alta tensão em build Release.

**PARA ESTUDAR**

- Normas IEC 60601-1, IEC 60601-2-10, ISO 14971 e IEC 62304 (verifique o acesso pela UFMG ou pela equipe)
- All About Circuits: Electrical Safety

**PRATIQUE**

**0.3.1** Liste três perigos de um eletroestimulador e, para cada um, uma mitigação em hardware, em firmware ou no app.

**0.3.2** Encontre na IEC 60601-2-10 os limites aplicáveis à saída de um estimulador e compare com os do Connect.

## 0.4 Sistemas numéricos e representação de dados

Binário, hexadecimal, complemento de dois, endianness e operações bit a bit. É a linguagem comum entre quem escreve firmware (registradores, protocolos) e quem escreve software (conversão de bytes).

> **NO CONNECT**
> Firmware e app trocam structs em binário. O `WeProtocol` no firmware e o pacote `byte_conversion` no app precisam concordar byte a byte.

**PARA ESTUDAR**

- SparkFun: Binary
- SparkFun: Hexadecimal
- Ben Eater no YouTube (vídeos sobre complemento de dois)

**PRATIQUE**

**0.4.1** Converta −1000 para int16 em complemento de dois e escreva os bytes em little-endian e em big-endian.

**0.4.2** Dado um registrador de 8 bits, escreva as máscaras para ligar o bit 3, desligar o bit 5 e testar o bit 7.

## 0.5 Linha de comando Linux

Shell, navegação, permissões, variáveis de ambiente e PATH. Quase toda ferramenta de engenharia (compiladores, build systems, SDKs) é usada a partir do terminal.

> **NO CONNECT**
> CMake e Ninja no firmware stimulus, `idf.py` no firmware BLE e o Flutter SDK com FVM no app: as três bases de código são compiladas pelo terminal.

**PARA ESTUDAR**

- The Missing Semester of Your CS Education (MIT)
- Linux Journey

**PRATIQUE**

0.5.1 Adicione uma pasta ao PATH de forma permanente e explique em qual arquivo a configuração ficou.
0.5.2 Escreva um script que compile um projeto e copie o binário gerado para uma pasta com a data no nome.

## 0.6 Git e fluxo de trabalho

Branches, merge e rebase, submódulos, mensagens no padrão Conventional Commits e code review. É como uma equipe trabalha no mesmo código sem pisar no trabalho do outro.

> **NO CONNECT**
> O firmware usa a pasta `Modules/` como submódulo compartilhado e o app depende de pacotes locais (`we_bluetooth`, `we_exception`...). Saber mover um submódulo de commit é rotina.

**PARA ESTUDAR**

- Pro Git (em português), capítulos 1 a 3 e a seção de submódulos
- Learn Git Branching
- Conventional Commits

**PRATIQUE**

0.6.1 Clone um repositório com submódulos, mude o submódulo de commit e explique o que aparece no `git status` do repositório pai.
0.6.2 Faça um rebase interativo juntando três commits em um, com mensagem no padrão Conventional Commits.
0.6.3 Revise o PR de um colega e deixe ao menos dois comentários construtivos.

## 0.7 Leitura de documentação técnica

Como ler datasheet, reference manual, esquemático e mapa de registradores: onde estão os limites absolutos, as condições típicas, os diagramas de tempo e os valores de reset. Ler com uma pergunta em mente economiza horas.

> **NO CONNECT**
> Os esquemáticos do Connect e os datasheets do ATSAMC20, do DAC53204 e do BQ25792 são as leituras de referência das trilhas de hardware e firmware.

**PARA ESTUDAR**

- SparkFun: How to Read a Schematic
- Datasheet do DAC53204 (Texas Instruments) como caso de estudo

**PRATIQUE**

0.7.1 No datasheet do DAC53204, encontre a tensão máxima de alimentação, o tempo de acomodação e o formato do quadro SPI.
0.7.2 Escolha um registrador do ATSAMC20 e explique cada campo e o seu valor de reset.

PARTE 01

# Hardware

Da lei de Ohm ao estágio de saída de alta tensão. A trilha ensina eletrônica analógica, digital e de potência de forma geral e usa os esquemáticos do Connect como material de estudo real.

## VOCÊ VAI APRENDER A

- √ Analisar e dimensionar circuitos analógicos e digitais de uma placa real.
- √ Entender fontes chaveadas, gate drivers, bateria e proteções.
- √ Planejar e executar a validação de um hardware em bancada.

## A TRILHA EM TRÊS NÍVEIS

### Básico

1.1 a 1.5, 5 tópicos

Circuitos, componentes, bancada e leitura de esquemático.

### Intermediário

1.6 a 1.12, 7 tópicos

Op-amps, fontes de corrente, conversores e chaves.

### Avançado

1.13 a 1.19, 7 tópicos

Potência, bateria, proteções, layout e validação.

**NÍVEL BÁSICO** 1.1 a 1.5, 5 tópicos

*Circuitos, componentes, bancada e leitura de esquemático.*

## 1.1 Circuitos elétricos

Lei de Ohm, leis de Kirchhoff, divisor de tensão, potência e dissipação. É a base para dimensionar qualquer componente de uma placa.

> **NO CONNECT**
> Divisores de tensão, resistores de sense e a dissipação dos componentes do estágio de saída são dimensionados com essas ferramentas.

**PARA ESTUDAR**

- SparkFun: Voltage, Current, Resistance, and Ohm's Law
- SparkFun: Voltage Dividers
- All About Circuits: textbook, volume de corrente contínua

**PRATIQUE**

1.1.1 Encontre um divisor de tensão no esquemático e calcule a tensão de saída e a corrente quiescente.
1.1.2 Calcule a potência dissipada no resistor de sense da fonte de corrente na corrente máxima de estímulo.

## 1.2 Componentes passivos

Resistor (potência e tolerância), capacitor (tipos, ESR, desacoplamento, perda de capacitância com tensão DC) e indutor (saturação e corrente RMS). Escolher o passivo certo é metade de um projeto confiável.

> **NO CONNECT**
> Cada capacitor dos esquemáticos tem um motivo para seu tipo e sua tensão nominal, principalmente perto dos trilhos de alta tensão.

**PARA ESTUDAR**

- SparkFun: Capacitors
- Livro: The Art of Electronics (Horowitz e Hill), capítulo 1

**PRATIQUE**

1.2.1 Escolha três capacitores do esquemático e justifique o tipo (cerâmico, eletrolítico) e a tensão nominal de cada um.
1.2.2 Explique por que um cerâmico X7R de 10 µF perde capacitância quando recebe tensão DC.

## 1.3 Semicondutores

Diodo, zener, LED, BJT (inclusive o par Darlington) e MOSFET como chave: regiões de operação, ganho, tensão de limiar e perdas.

> **NO CONNECT**
> O estágio de saída usa transistores como o Darlington FJD5304D. Entender saturação e ganho mínimo é o que garante que eles conduzam a corrente prevista.

**PARA ESTUDAR**

- SparkFun: Diodes
- SparkFun: Transistors
- Datasheet do FJD5304D

**PRATIQUE**

1.3.1 Calcule a corrente de base necessária para saturar um BJT do esquemático a partir do ganho mínimo do datasheet.
1.3.2 Compare o uso de um MOSFET e de um Darlington como chave em alta tensão.

## 1.4 Instrumentação de bancada

Multímetro, osciloscópio (trigger, acoplamento, largura de banda), fonte de bancada com limite de corrente, carga resistiva e ponteiras de alta tensão e diferenciais. Medir errado é pior que não medir.

> **NO CONNECT**
> Medir pulsos no estágio de saída exige carga resistiva e cuidado com a referência: GND e GND_HV não são o mesmo ponto.

**PARA ESTUDAR**

- SparkFun: How to Use a Multimeter
- SparkFun: How to Use an Oscilloscope
- EEVblog no YouTube

**PRATIQUE**

1.4.1 Meça um pulso bifásico numa carga resistiva e registre amplitude, largura e intervalo.
1.4.2 Explique por que não se deve prender a referência da ponteira no GND_HV sem isolação ou ponteira diferencial.

## 1.5 Leitura de esquemático

Hierarquia de páginas, net labels, designators, notas de montagem e grounds separados. Saber navegar num esquemático grande é a habilidade mais usada no dia a dia.

> **NO CONNECT**
> Os esquemáticos `SCH070105` (CFCU) e `SCH070102` (CTCE) têm várias páginas e dois grounds, GND e GND_HV.

**PARA ESTUDAR**

- SparkFun: How to Read a Schematic
- Esquemáticos SCH070105 e SCH070102

**PRATIQUE**

**1.5.1** Siga um sinal de controle desde o pino do ATSAM até o estágio de saída, listando cada página por onde ele passa.

**1.5.2** Liste onde GND e GND_HV se encontram (ou não) e explique o motivo.

**NÍVEL INTERMEDIÁRIO** 1.6 a 1.12, 7 tópicos

*Op-amps, fontes de corrente, conversores e chaves.*

## 1.6 Amplificadores operacionais

Op-amp ideal e real (offset, GBW, slew rate, saída rail-to-rail), buffer, inversor e não inversor, comparador, histerese e Schmitt trigger.

> **NO CONNECT**
> Op-amps aparecem como fonte de corrente, como comparadores das proteções e como buffers de sinal ao longo das placas.

**PARA ESTUDAR**

- Livro: Op Amps for Everyone (Texas Instruments, Ron Mancini), gratuito
- All About Circuits: textbook, capítulo de op-amps

**PRATIQUE**

- **1.6.1** Projete um comparador com histerese de 100 mV e simule no LTspice.
- **1.6.2** Encontre no esquemático um op-amp em cada configuração (buffer, não inversor, comparador) e calcule o ganho ou os limiares.

## 1.7 Fontes de corrente controladas por tensão

Um op-amp, um transistor e um resistor de sense formam uma malha que força a tensão no shunt a seguir a referência. O resultado é I = Vref / Rsense, independente da carga até o limite de compliance. É o coração de qualquer estimulador por corrente.

> **NO CONNECT**
> No Connect, a fonte de corrente é o conjunto TLV9151 + Q36 + R196, com a referência vinda do DAC.

**PARA ESTUDAR**

- Livro: The Art of Electronics, seção de fontes de corrente com op-amp
- Datasheet do TLV9151

**PRATIQUE**

- **1.7.1** Calcule a corrente de saída para Vref = 1 V com o valor de R196 do esquemático.
- **1.7.2** Encontre a tensão de compliance: até que impedância de carga a fonte mantém a corrente com +125 V?
- **1.7.3** Simule o circuito no LTspice com uma carga RC representando pele e eletrodo.

## 1.8 Medição de corrente

Resistor shunt, amplificador de sense de corrente e comparador com latch para desligamento por sobre-corrente. Medir corrente com precisão é o que permite controlá-la e limitá-la.

> **NO CONNECT**
> O INA310 combina amplificador de sense e comparador, e alimenta a proteção de sobrecorrente.

**PARA ESTUDAR**

- Datasheet do INA310 (Texas Instruments)
- TI Precision Labs, série de vídeos sobre current sensing (site da TI)

**PRATIQUE**

1.8.1 Calcule a tensão de saída do amplificador de sense na corrente máxima e verifique se cabe na faixa do ADC.
1.8.2 Explique por que o comparador de proteção precisa travar (latch) em vez de apenas comparar.

## 1.9 Lógica digital

Portas lógicas, buffers e inversores Schmitt, saídas open-drain com pull-up e compatibilidade de níveis lógicos entre 3,3 V e 5 V.

> **NO CONNECT**
> Buffers e inversores como o 74LVC1G14 e o 74LVC1G17 limpam sinais que cruzam a placa e montam a lógica de desligamento.

**PARA ESTUDAR**

- SparkFun: Logic Levels
- SparkFun: Pull-up Resistors
- Datasheets 74LVC1G14 e 74LVC1G17

**PRATIQUE**

1.9.1 Dimensione o pull-up de uma linha open-drain para um tempo de subida de 1 µs com 50 pF.
1.9.2 Explique por que usar um buffer Schmitt num sinal que vem de outra placa.

## 1.10 Conversores DAC e ADC

Resolução, LSB, tensão de referência (VDD, interna ou externa), erros de offset e ganho, INL e DNL. Conversores são a fronteira entre o mundo digital do firmware e o analógico do circuito.

> **NO CONNECT**
> O DAC53204 define a referência da fonte de corrente, ou seja, a amplitude do estímulo.

**PARA ESTUDAR**

› SparkFun: Analog to Digital Conversion
› Datasheet do DAC53204 (Texas Instruments)

**PRATIQUE**

1.10.1 Calcule o passo de corrente na saída correspondente a 1 LSB do DAC.
1.10.2 Compare a precisão da saída usando referência VDD e referência interna.

---

## 1.11 Chaves analógicas e multiplexação

Resistência on, carga injetada e faixa de sinal das chaves analógicas, usadas para rotear um sinal entre caminhos diferentes.

> **NO CONNECT**
> O NLAS4157 compartilha o sinal entre os modos FES e tDCS.

**PARA ESTUDAR**

› Datasheet do NLAS4157 (onsemi)

**PRATIQUE**

1.11.1 Encontre o NLAS4157 no esquemático e descreva quais sinais ele seleciona e quem o controla.
1.11.2 Estime o erro introduzido pela resistência on da chave no caminho do sinal.

---

## 1.12 Relés de estado sólido e isolação

Relés PhotoMOS: LED de entrada, MOSFETs de saída, isolação galvânica e acionamento por transistor. Permitem abrir um caminho de alta tensão a partir de um sinal lógico.

> **NO CONNECT**
> O AQW280SX é acionado via transistor e abre o caminho de saída quando necessário.

**PARA ESTUDAR**

› Datasheet do AQW280SX (Panasonic)

**PRATIQUE**

1.12.1 Calcule o resistor de entrada do relé para a corrente de LED recomendada.
1.12.2 Explique qual falha o relé previne quando aberto e o que acontece se ele colar fechado.

**NÍVEL AVANÇADO** 1.13 a 1.19, 7 tópicos

*Potência, bateria, proteções, layout e validação.*

## 1.13 Fontes chaveadas

Topologias buck, boost e push-pull, malha de controle, transformador e retificação. É como se gera, com eficiência, uma tensão muito maior ou menor que a da bateria.

> **NO CONNECT**
> Os +125 V do estágio de saída vêm de um conversor push-pull com o controlador LM5030.

**PARA ESTUDAR**

- Livro: Fundamentals of Power Electronics (Erickson e Maksimović), capítulos 1 a 6
- Nota de aplicação TI SLVA477: Basic Calculation of a Buck Converter's Power Stage
- Datasheet do LM5030

**PRATIQUE**

1.13.1 Desenhe o caminho de energia da bateria até os +125 V, indicando cada conversor.
1.13.2 Calcule o ciclo de trabalho e a relação de espiras esperados no push-pull.

## 1.14 Gate drivers e ponte H

Half-bridge e ponte H, gate drivers com bootstrap e seus limites: o capacitor de bootstrap precisa ser recarregado, então não sustenta condução contínua indefinidamente. A ponte H permite inverter a polaridade da corrente na carga.

> **NO CONNECT**
> O IRS25752 aciona o estágio bifásico do FES. O tDCS, por ser corrente contínua, exige outra solução.

**PARA ESTUDAR**

- Infineon (IR) AN-978: HV Floating MOS-Gate Driver ICs
- Datasheet do IRS25752

**PRATIQUE**

1.14.1 Calcule o capacitor de bootstrap mínimo para a maior largura de pulso usada.
1.14.2 Explique por que corrente contínua de longa duração não combina com bootstrap.

## 1.15 Gerenciamento de bateria

Química Li-ion (carga CC/CV, limites de temperatura), carregadores controlados por I²C, power path e proteções. Bateria mal gerenciada é risco de segurança, não só de autonomia.

> **NO CONNECT**
> O carregador é o BQ25792, configurado pelo firmware via I²C.

**PARA ESTUDAR**

- Battery University
- Datasheet do BQ25792 (Texas Instruments)

**PRATIQUE**

1.15.1 Liste os registradores do BQ25792 que definem corrente de carga, tensão final e limite de entrada.
1.15.2 Explique o que o power path faz quando o carregador está conectado e a bateria está descarregada.

## 1.16 Sequenciamento de energia e load switches

Habilitação ordenada dos trilhos de alimentação, corrente de inrush, soft-start e descarga ativa ao desligar. A ordem em que a placa liga define se ela liga direito.

> **NO CONNECT**
> Os trilhos do Connect, inclusive o de alta tensão, são habilitados em sequência.

**PARA ESTUDAR**

- Notas de aplicação da TI sobre load switches (site da TI)

**PRATIQUE**

1.16.1 Monte a tabela de sequência de ligação dos trilhos a partir do esquemático.
1.16.2 Meça com o osciloscópio a subida de dois trilhos e compare com a ordem esperada.

## 1.17 Proteções de hardware

Sobrecorrente, parada de emergência, detecção de eletrodo solto e uma lógica OR que desliga a saída sem depender do firmware. Em equipamento médico, a última linha de defesa precisa ser física.

> **NO CONNECT**
> As proteções do Connect se juntam numa lógica OR de desligamento, documentada nas páginas de proteção do `SCH070102`.

**PARA ESTUDAR**

› IEC 60601-2-10, requisitos de proteção de estimuladores
› Esquemático SCH070102, páginas de proteção

**PRATIQUE**

1.17.1 Desenhe a lógica OR de desligamento: quais entradas, qual saída e o que ela comanda.
1.17.2 Para cada proteção, indique o tempo de resposta esperado e quem rearma o sistema.

## 1.18 Integridade de sinal e layout

Desacoplamento, laços de corrente, planos de referência, distâncias de isolação (creepage e clearance) e EMC. Um esquemático certo pode falhar por causa de um layout ruim.

> **NO CONNECT**
> A placa convive com +125 V e lógica de 3,3 V, então as distâncias de isolação seguem a IEC 60601-1.

**PARA ESTUDAR**

› Phil's Lab no YouTube
› Palestras de Rick Hartley sobre aterramento e retorno de corrente (YouTube)
› Livro: Electromagnetic Compatibility Engineering (Henry Ott)

**PRATIQUE**

1.18.1 No layout, encontre o laço de maior di/dt do conversor e proponha como reduzi-lo.
1.18.2 Verifique a distância de isolação entre a região de alta tensão e a de baixa tensão e compare com a tabela da IEC 60601-1.

## 1.19 Validação de hardware

Plano de teste, teste de disparo de cada proteção e caracterização da saída (linearidade, erro, ripple). Validar é provar, com dados, que o hardware faz o que o projeto promete.

> **NO CONNECT**
> A caracterização principal é a curva corrente de saída × código do DAC, medida em carga resistiva.

**PARA ESTUDAR**

› Modelo de plano de teste da equipe (a criar)

**PRATIQUE**

1.19.1 Escreva um plano de teste para a proteção de sobrecorrente: setup, passos e critério de aprovação.
1.19.2 Levante a curva corrente de saída × código do DAC e calcule o erro máximo.

PARTE 02

# Firmware

Programação de microcontroladores do registrador ao sistema completo: C e C++ embarcado, periféricos, protocolos, arquitetura, tempo real, Bluetooth e segurança. Os dois processadores do Connect, um ATSAMC20 bare-metal e um ESP32 com FreeRTOS, servem de laboratório.

## VOCÊ VAI APRENDER A

- √ Escrever código embarcado seguro, sem heap e sem bloqueio, em C e C++.
- √ Configurar periféricos, escrever drivers e depurar direto no hardware.
- √ Projetar arquitetura, protocolos e testes de um firmware crítico.

## A TRILHA EM TRÊS NÍVEIS

### Básico

2.1 a 2.6, 6 tópicos

C, C++ embarcado, microcontrolador, GPIO e ferramentas de build.

### Intermediário

2.7 a 2.14, 8 tópicos

Cortex-M, periféricos, protocolos seriais, drivers e depuração.

### Avançado

2.15 a 2.29, 15 tópicos

Arquitetura, tempo real, BLE, protocolo próprio, segurança e testes.

**NÍVEL BÁSICO** 2.1 a 2.6, 6 tópicos

*C, C++ embarcado, microcontrolador, GPIO e ferramentas de build.*

## 2.1 Linguagem C

Tipos de largura fixa (`uint8_t`, `int16_t`), ponteiros, structs, `volatile`, `static`, bitfields e máscaras. É a base de qualquer código que toca registradores.

> **NO CONNECT**
> Mesmo com o firmware escrito em C++20, a interface com o hardware (registradores e código gerado pelo Harmony) é C puro.

**PARA ESTUDAR**

- Beej's Guide to C Programming
- Livro: The C Programming Language (Kernighan e Ritchie)

**PRATIQUE**

2.1.1 Escreva uma função que empacote três valores (4, 6 e 6 bits) num `uint16_t` e outra que os desempacote.
2.1.2 Mostre, com um exemplo, o que pode acontecer se uma flag alterada numa interrupção não for `volatile`.

## 2.2 C++ para embarcados

Classes, namespaces, `enum class`, `constexpr`, templates, `std::array` e `std::span` e RAII, usados sem exceções, sem RTTI e sem alocação dinâmica. C++ bem usado dá segurança de tipos sem custo de desempenho.

> **NO CONNECT**
> O `ergo_fes_connect_stimulus_firmware` é C++20 bare-metal, sem exceções, sem RTTI e sem heap.

**PARA ESTUDAR**

- cppreference
- Embedded Artistry, artigos sobre C++ em embarcados

**PRATIQUE**

2.2.1 Reescreva um conjunto de `#define` de estados como `enum class` e mostre o erro de compilação que isso passa a evitar.
2.2.2 Implemente um guard RAII que desabilita interrupções no construtor e as restaura no destrutor.

## 2.3 Arquitetura de microcontroladores

CPU, memória flash e RAM, barramento, periféricos mapeados em memória e árvore de clock. Entender o mapa do chip é entender o que o código realmente faz.

> **NO CONNECT**
> O processador de estímulo é o ATSAMC20N17A, um Cortex-M0+ da Microchip.

**PARA ESTUDAR**

- Quantum Leaps (Miro Samek): Modern Embedded Systems Programming
- Datasheet do SAM C20/C21 (Microchip), visão geral e mapa de memória

**PRATIQUE**

2.3.1 A partir do datasheet, desenhe o mapa de memória do ATSAMC20N17A (flash, RAM, periféricos).
2.3.2 Explique como o clock chega a um SERCOM: fonte, gerador de clock e habilitação.

## 2.4 GPIO

Entrada, saída, pull-up interno, leitura de botões com debounce e acesso direto aos registradores de porta. O primeiro contato real com o hardware.

> **NO CONNECT**
> No ATSAMC20 os pinos são controlados pelo periférico PORT.

**PARA ESTUDAR**

- SparkFun: Pull-up Resistors
- Datasheet do SAM C20, capítulo PORT

**PRATIQUE**

2.4.1 Pisque um LED escrevendo direto nos registradores do PORT, sem bibliotecas.
2.4.2 Implemente um debounce não bloqueante, baseado em tempo, para um botão.

## 2.5 Toolchain

Compilador, assembler e linker; `arm-none-eabi-gcc`, arquivos objeto, ELF, binário e map file. Saber o que cada etapa produz ajuda a entender erros de build e problemas de memória.

> **NO CONNECT**
> O firmware stimulus é compilado com `arm-none-eabi-gcc` e gera um ELF com map file.

**PARA ESTUDAR**

› Memfault Interrupt: From Zero to main()

**PRATIQUE**

2.5.1 Compile o firmware e use `arm-none-eabi-size` para ver quanto de flash e RAM é usado.
2.5.2 Abra o map file e encontre as três funções que mais ocupam flash.

---

## 2.6 Build system

CMake, Ninja, toolchain files e `compile_commands.json`, o arquivo que alimenta clangd e clang-tidy. Um build reprodutível é o que permite que qualquer pessoa compile o projeto.

> **NO CONNECT**
> O firmware stimulus usa CMake e Ninja com um toolchain file para ARM.

**PARA ESTUDAR**

› Tutorial oficial do CMake

**PRATIQUE**

2.6.1 Configure e compile o firmware do zero numa pasta de build nova, nas configurações Debug e Release.
2.6.2 Explique o que o toolchain file do projeto define e por que ele é necessário.

**NÍVEL INTERMEDIÁRIO** 2.7 a 2.14, 8 tópicos

*Cortex-M, periféricos, protocolos seriais, drivers e depuração.*

## 2.7 Cortex-M0+

Registradores do núcleo, modos de execução, NVIC, interrupções e prioridades, exceções e SysTick. É o comportamento comum a todos os microcontroladores dessa família.

> **NO CONNECT**
> O ATSAMC20 é um Cortex-M0+; tudo deste tópico vale diretamente para ele.

**PARA ESTUDAR**

- ARM: Cortex-M0+ Devices Generic User Guide
- Livro: The Definitive Guide to ARM Cortex-M0 and Cortex-M0+ Processors (Joseph Yiu)

**PRATIQUE**

2.7.1 Configure o SysTick para gerar uma interrupção a cada 1 ms e conte o tempo desde o boot.
2.7.2 Explique o que acontece quando duas interrupções de mesma prioridade chegam ao mesmo tempo.

## 2.8 Startup e linker script

Tabela de vetores, handler de reset, cópia de `.data`, zeragem de `.bss` e o linker script que diz onde cada coisa mora na memória. É o que acontece antes do `main()`.

> **NO CONNECT**
> O firmware stimulus tem seu próprio `Startup.c` e arquivo `.ld`.

**PARA ESTUDAR**

- Memfault Interrupt: série From Zero to main()

**PRATIQUE**

2.8.1 Leia o `Startup.c` do firmware e descreva cada passo até chegar ao `main()`.
2.8.2 Crie uma variável numa seção própria pelo linker script e confirme o endereço no map file.

## 2.9 Periféricos de um microcontrolador

Portas, interfaces seriais configuráveis (SPI, I²C, USART), timers, ADC, EEPROM e controlador de memória não volátil. Cada periférico tem registradores de configuração, de status e de dados.

> **NO CONNECT**
> No ATSAMC20: PORT, SERCOM (SPI, I²C, USART), TC (timers), ADC, EEPROM e NVM.

**PARA ESTUDAR**

› Datasheet do SAM C20/C21 (Microchip), capítulos de cada periférico

**PRATIQUE**

2.9.1 Liste quais SERCOMs o firmware usa, em qual modo e para qual CI externo.
2.9.2 Configure um timer para gerar uma onda quadrada de 1 kHz num pino e meça no osciloscópio.

## 2.10 Geradores de código de fabricante

Ferramentas como MPLAB Harmony/MCC geram o BSP e as bibliotecas de periféricos (PLIBs) a partir de uma configuração gráfica. A regra de ouro: código gerado não se edita à mão; muda-se a configuração e regenera.

> **NO CONNECT**
> O firmware stimulus usa MPLAB Harmony/MCC, com fronteira clara entre o código gerado e o código da equipe.

**PARA ESTUDAR**

› MPLAB Harmony no GitHub (repositórios csp e documentação)

**PRATIQUE**

2.10.1 Altere a frequência de um SPI pelo MCC, regenere e mostre o diff resultante.
2.10.2 Explique onde fica a fronteira entre código gerado e código da equipe no repositório.

## 2.11 Protocolos seriais

SPI, I²C e UART: modos de clock, endereçamento, framing e checksum. São as estradas por onde um microcontrolador conversa com CIs e com outros processadores.

> **NO CONNECT**
> SPI fala com o DAC, I²C com o carregador e a UART liga o ATSAM ao ESP32.

**PARA ESTUDAR**

› SparkFun: Serial Peripheral Interface (SPI)
› SparkFun: I2C
› SparkFun: Serial Communication

**PRATIQUE**

2.11.1 Capture com um analisador lógico uma escrita SPI no DAC e decodifique o quadro.
2.11.2 Capture um quadro na UART entre ATSAM e ESP32 e identifique cabeçalho, payload e checksum.

## 2.12 Drivers de CIs externos

Como escrever um driver a partir do mapa de registradores: camada de transporte, definição de registradores, API de alto nível e tratamento de erro. Um bom driver esconde o protocolo e expõe intenções.

> **NO CONNECT**
> Os drivers do DAC53204 e do BQ25792 são os modelos de referência da equipe.

**PARA ESTUDAR**

› Datasheets do DAC53204 e do BQ25792
› Drivers existentes no repositório

**PRATIQUE**

2.12.1 Leia o driver do DAC53204 e desenhe a sequência de comandos de inicialização.
2.12.2 Implemente a leitura de um registrador de status do BQ25792 que ainda não tenha função no driver.

## 2.13 Gravação e depuração

SWD, J-Link, OpenOCD e GDB: breakpoints, watchpoints e inspeção de registradores e memória. Depurador bem usado substitui horas de `printf`.

> **NO CONNECT**
> O ATSAMC20 é gravado e depurado por SWD.

**PARA ESTUDAR**

› SEGGER Wiki
› OpenOCD
› Documentação do GDB

**PRATIQUE**

2.13.1 Grave o firmware, pare num breakpoint dentro de um Handler e inspecione uma variável e um registrador de periférico.
2.13.2 Use um watchpoint para descobrir quem altera um campo do `SystemData`.

## 2.14 Qualidade de código

clang-format, clang-tidy, warnings tratados como erros, padrão de nomes e comentários com autor. Qualidade é o que torna o código revisável por outra pessoa.

> **NO CONNECT**
> O firmware usa `.clang-format`, clang-tidy e warnings como erros.

**PARA ESTUDAR**

- ClangFormat
- clang-tidy
- Guia de estilo interno do repositório

**PRATIQUE**

2.14.1 Rode clang-tidy no projeto e corrija dois avisos, explicando cada um.
2.14.2 Configure o editor para formatar ao salvar usando o `.clang-format` do repositório.

**NÍVEL AVANÇADO** 2.15 a 2.29, 15 tópicos

*Arquitetura, tempo real, BLE, protocolo próprio, segurança e testes.*

## 2.15 Arquitetura superloop cooperativa

Services com `Init`, `DeInit` e `Handler` chamados em sequência no laço principal. Funciona desde que nenhum Handler bloqueie: cada um tem um orçamento de latência.

> **NO CONNECT**
> O firmware stimulus é organizado assim; o `main` chama os Handlers de todos os services a cada volta.

**PARA ESTUDAR**

- Livro: Making Embedded Systems (Elecia White), capítulos de arquitetura
- Código do main e dos services no firmware stimulus

**PRATIQUE**

2.15.1 Meça com um GPIO e o osciloscópio o tempo de uma volta do superloop e o do Handler mais lento.
2.15.2 Converta um trecho com espera ativa (`delay`) numa versão não bloqueante.

## 2.16 Máquinas de estados

Modelagem de estados e transições, implementação com `switch` ou tabela, e diagramas. É a forma mais clara de descrever comportamento que depende do histórico.

> **NO CONNECT**
> O `StimulusService` e o `SystemController` são máquinas de estados.

**PARA ESTUDAR**

- Livro: Practical UML Statecharts in C/C++ (Miro Samek)
- Mermaid: diagramas de estado

**PRATIQUE**

2.16.1 Desenhe o diagrama de estados do `StimulusService` a partir do código.
2.16.2 Encontre uma transição não tratada, ou justifique por que todas estão.

## 2.17 Temporização precisa

Base de tempo em microssegundos, geração de pulsos com largura e intervalo exatos e medição de jitter. Em estimulação, tempo é parâmetro clínico.

> **NO CONNECT**
> O `MicroTimerService` fornece a base de µs usada para gerar os pulsos bifásicos.

**PARA ESTUDAR**

- Datasheet do SAM C20, capítulo TC
- Código do MicroTimerService

**PRATIQUE**

2.17.1 Meça o jitter da largura do pulso em 1000 pulsos e relacione com a carga do superloop.
2.17.2 Explique como o firmware trata o overflow do contador de µs.

## 2.18 Concorrência com interrupções

Seções críticas, condições de corrida, acesso atômico a variáveis multibyte e buffers circulares entre interrupção e laço principal. Bugs de concorrência são raros, intermitentes e caros.

> **NO CONNECT**
> Num superloop, dados que chegam por interrupção, como os bytes da UART, precisam de um caminho seguro até o laço principal.

**PARA ESTUDAR**

- Embedded Artistry: Creating a Circular Buffer in C and C++
- Memfault Interrupt, artigos sobre interrupções no Cortex-M

**PRATIQUE**

2.18.1 Implemente um buffer circular de um produtor e um consumidor sem desabilitar interrupções e justifique por que é seguro.
2.18.2 Crie de propósito uma condição de corrida numa variável de 32 bits lida em duas partes e reproduza o erro.

## 2.19 Tratamento de erros

Propagação de erro sem exceções, guards de escopo e transição para estados de falha seguros. Todo erro precisa de um destino definido.

> **NO CONNECT**
> O firmware usa o tipo `ReturnStatus` e guards de escopo.

**PARA ESTUDAR**

› Código de ReturnStatus e dos guards no repositório

**PRATIQUE**

2.19.1 Rastreie o que acontece desde uma falha de escrita SPI no DAC até o estado do sistema.
2.19.2 Proponha um código de erro novo e liste todos os lugares que precisariam tratá-lo.

---

## 2.20 Persistência

EEPROM e memória não volátil, layout versionado dos dados, escrita atrasada para poupar ciclos e desgaste. Dados persistidos sobrevivem a versões de firmware, então precisam de versão.

> **NO CONNECT**
> O firmware stimulus persiste dados em EEPROM usando escrita atrasada.

**PARA ESTUDAR**

› Datasheet do SAM C20, capítulos NVMCTRL e RWW EEPROM

**PRATIQUE**

2.20.1 Estime quantos anos a EEPROM dura com a política atual de escrita.
2.20.2 Explique como o firmware deve reagir a dados gravados por uma versão antiga.

---

## 2.21 Estado central e sincronização

Uma fonte única do estado do dispositivo e a forma de espelhá-lo em outros processadores. Evita que cada módulo tenha sua própria versão da verdade.

> **NO CONNECT**
> O singleton `SystemData` concentra o estado, e os valores são espelhados no ESP32 para leitura via BLE.

**PARA ESTUDAR**

› Código do SystemData e da camada de sincronização

**PRATIQUE**

2.21.1 Liste os campos do `SystemData` e, para cada um, quem escreve e quem lê.
2.21.2 Desenhe a sequência de uma mudança no nível da bateria até o valor chegar ao app.

## 2.22 Protocolo binário próprio

Framing, serialização de structs, identificadores de mensagem, versionamento e regras de compatibilidade. Um protocolo vive mais que qualquer versão de firmware ou de app.

> **NO CONNECT**
> O `WeProtocol` liga ATSAM, ESP32 e app, com wire IDs que precisam permanecer compatíveis.

**PARA ESTUDAR**

› Documentação e código do WeProtocol no repositório

**PRATIQUE**

2.22.1 Monte manualmente, byte a byte, um quadro de comando e confira com uma captura real.
2.22.2 Adicione um campo a uma mensagem sem quebrar app ou firmware antigos e explique a estratégia.

## 2.23 ESP32 e ESP-IDF

Estrutura de projeto ESP-IDF (componentes, `main`), `idf.py`, `sdkconfig` e menuconfig, tabela de partições e logs.

> **NO CONNECT**
> O `ergo_fes_connect_ble_firmware` é um projeto ESP-IDF.

**PARA ESTUDAR**

› Documentação do ESP-IDF, seção Get Started

**PRATIQUE**

2.23.1 Compile e grave o firmware BLE e acompanhe o boot pelo monitor serial.
2.23.2 Mude uma opção pelo menuconfig e mostre o diff no `sdkconfig`.

## 2.24 FreeRTOS

Tasks e prioridades, escalonador preemptivo, filas, semáforos, mutexes e callbacks. Um RTOS troca a previsibilidade do superloop por concorrência real.

> **NO CONNECT**
> O firmware BLE roda sobre FreeRTOS, integrado ao ESP-IDF.

**PARA ESTUDAR**

› FreeRTOS.org e o livro gratuito Mastering the FreeRTOS Real Time Kernel

**PRATIQUE**

2.24.1 Liste as tasks do firmware BLE com prioridade e tamanho de stack, e meça o stack livre de cada uma.
2.24.2 Explique como um dado sai da task da UART e chega à pilha BLE sem condição de corrida.

---

## 2.25 Bluetooth Low Energy: lado periférico

GAP e advertising, GATT (services, characteristics, descriptor CCC), notify e indicate, MTU. É como um dispositivo se anuncia e expõe dados para um celular.

> **NO CONNECT**
> O ESP32 do Connect é o periférico BLE e usa a pilha Bluedroid do ESP-IDF.

**PARA ESTUDAR**

› Nordic DevAcademy: Bluetooth Low Energy Fundamentals
› Novel Bits, artigos introdutórios de BLE
› Exemplos gatt_server do ESP-IDF

**PRATIQUE**

2.25.1 Conecte-se com o nRF Connect, liste services e characteristics do Connect e associe cada UUID à sua função.
2.25.2 Explique a diferença prática entre notify e indicate e onde cada um é usado no projeto.

---

## 2.26 Ponte entre BLE e UART

Roteamento de comandos entre characteristics BLE e quadros seriais, validação de toda entrada vinda do celular e limites da tabela de atributos.

> **NO CONNECT**
> O firmware BLE faz exatamente essa ponte entre o app e o ATSAM.

**PARA ESTUDAR**

› Código da ponte no firmware BLE

**PRATIQUE**

2.26.1 Desenhe o caminho de uma escrita BLE até o quadro UART correspondente.
2.26.2 Envie pelo nRF Connect um payload malformado e verifique se o firmware o rejeita.

## 2.27 Código compartilhado entre plataformas

Bibliotecas comuns como submódulo e uma camada de abstração de plataforma, para que o mesmo código rode em microcontroladores diferentes e até no PC.

> **NO CONNECT**
> A pasta `Modules/` é compartilhada entre os firmwares, com abstração para ATSAM, ESP e STM32.

**PARA ESTUDAR**

› Pasta Modules/ do repositório

**PRATIQUE**

2.27.1 Identifique o que, em `Modules/`, depende de plataforma e como essa dependência é injetada.
2.27.2 Compile um módulo comum para o PC com um stub de plataforma.

## 2.28 Firmware de segurança crítica

Defesa em profundidade, watchdog, falha segura, interação com as proteções de hardware e diferenças entre builds de depuração e de produção. O firmware nunca é a única barreira, mas precisa ser uma boa barreira.

> **NO CONNECT**
> No firmware stimulus, a alta tensão só é habilitada em build Release (com `NDEBUG`).

**PARA ESTUDAR**

› IEC 62304 (ciclo de vida de software médico)
› Livro: Embedded Software Development for Safety-Critical Systems (Chris Hobbs)

**PRATIQUE**

2.28.1 Liste o que impede o firmware de ligar a alta tensão num build Debug e confirme no código.
2.28.2 Simule um travamento do superloop e verifique se o watchdog leva o sistema a um estado seguro.

## 2.29 Testes de firmware

Testes unitários no PC, testes de bancada, regressão e rastreabilidade entre requisito, código e teste. Firmware testado no PC é firmware que pode mudar sem medo.

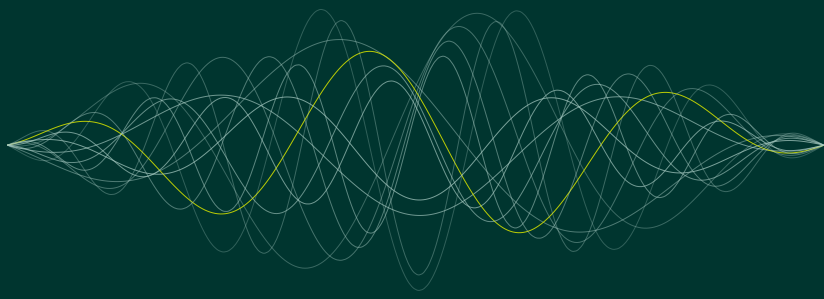
> **NO CONNECT**
> A lógica pura (protocolo, máquinas de estados) pode ser testada no PC; o resto, em bancada.

#### PARA ESTUDAR

- GoogleTest
- Unity
- CppUTest
- Livro: Test-Driven Development for Embedded C (James Grenning)

#### PRATIQUE

2.29.1 Escreva testes no PC para a serialização de uma mensagem do `WeProtocol`.
2.29.2 Escreva um roteiro de teste de bancada para a sequência de início e parada de estímulo.



PARTE 03

# Software

Desenvolvimento de aplicativos móveis com Flutter, do Dart básico à comunicação com dispositivos por Bluetooth, backend e lojas de aplicativo. O app do Connect é o caso de estudo do começo ao fim.

## VOCÊ VAI APRENDER A

- √ Construir telas, estado e navegação em Flutter com arquitetura em camadas.
- √ Falar com um dispositivo por BLE respeitando o contrato com o firmware.
- √ Testar, integrar com backend e publicar um app de forma confiável.

## A TRILHA EM TRÊS NÍVEIS

### Básico

3.1 a 3.4, 4 tópicos

Dart, Flutter, ambiente e projetos Android e iOS.

### Intermediário

3.5 a 3.13, 9 tópicos

Estado, módulos, camadas, pacotes, dados e qualidade.

### Avançado

3.14 a 3.24, 11 tópicos

BLE, bytes, backend, fluxos clínicos, testes e publicação.

**NÍVEL BÁSICO** 3.1 a 3.4, 4 tópicos

*Dart, Flutter, ambiente e projetos Android e iOS.*

## 3.1 Linguagem Dart

Tipos, null safety, classes e mixins, `async`/`await`, `Future` e `Stream`. Dart é a linguagem do Flutter e tem um modelo assíncrono que aparece em todo lugar.

> **NO CONNECT**
> Dados que chegam do dispositivo de forma contínua, como bateria e estado, são modelados naturalmente como `Stream`.

**PARA ESTUDAR**

- Dart language tour
- Sound null safety
- DartPad, para testar no navegador

**PRATIQUE**

3.1.1 Escreva um `Stream` que emita a contagem regressiva de um exercício de 10 s e consuma com `await for`.
3.1.2 Reescreva uma classe com campos anuláveis para que o compilador garanta a inicialização.

## 3.2 Fundamentos de Flutter

Widgets, árvore de widgets, Stateless e Stateful, ciclo de vida, layout (Row, Column, Flex) e navegação. Em Flutter, toda a interface é uma árvore de widgets reconstruída a partir do estado.

> **NO CONNECT**
> Todas as telas do `app_main` são composições de widgets sobre esses fundamentos.

**PARA ESTUDAR**

- Documentação do Flutter
- Flutterando (comunidade brasileira) no YouTube

**PRATIQUE**

3.2.1 Monte uma tela de controle com slider de amplitude e botão iniciar/parar, usando só estado local.
3.2.2 Explique quando o `build` é chamado e como evitar reconstruções desnecessárias.

## 3.3 Ambiente de desenvolvimento

Flutter SDK, FVM para fixar a versão, `pubspec.yaml`, pacotes e Android Studio/Xcode. Um ambiente reprodutível evita o clássico “na minha máquina funciona”.

> **NO CONNECT**
> O projeto fixa o Flutter 3.19.6 no `.fvmrc`.

**PARA ESTUDAR**

› FVM
› pub.dev

**PRATIQUE**

3.3.1 Configure o FVM, rode o `app_main` num emulador e anote cada problema encontrado (vira FAQ deste documento).
3.3.2 Explique a diferença entre `pubspec.yaml` e `pubspec.lock` e qual deles vai para o git.

## 3.4 Projetos Android e iOS

Estrutura das pastas `android/` e `ios/`, permissões, assinatura e build de release. Por baixo do Flutter continuam existindo dois projetos nativos.

> **NO CONNECT**
> Escanear e conectar a dispositivos BLE exige permissões específicas em cada plataforma; confira as que o projeto declara.

**PARA ESTUDAR**

› Flutter: deployment

**PRATIQUE**

3.4.1 Liste as permissões de Bluetooth declaradas em Android e iOS e explique por que cada uma existe.
3.4.2 Gere um APK de release assinado com uma chave de teste.

**NÍVEL INTERMEDIÁRIO** 3.5 a 3.13, 9 tópicos

*Estado, módulos, camadas, pacotes, dados e qualidade.*

## 3.5 Gerência de estado com BLoC

`flutter_bloc`: eventos, estados, Bloc e Cubit. Separa a lógica da interface e torna o comportamento previsível e testável.

> **NO CONNECT**
> O app usa `flutter_bloc`, com Blocs e Cubits.

**PARA ESTUDAR**

› bloclibrary.dev

**PRATIQUE**

3.5.1 Refaça a tela do exercício 3.2 com um Cubit.
3.5.2 Desenhe o fluxo evento → estado de um Bloc real do app.

## 3.6 Modularização e injeção de dependência

Dividir o app em módulos com rotas e dependências próprias, e injetar serviços em vez de criá-los à mão. Facilita trabalho em paralelo e testes.

> **NO CONNECT**
> O app usa `flutter_modular`, com módulos como `fes`, `breath`, `devices`, `patient` e `medic`.

**PARA ESTUDAR**

› Documentação do flutter_modular

**PRATIQUE**

3.6.1 Mapeie os módulos do app e as rotas de cada um.
3.6.2 Crie um módulo vazio com uma rota e um bind e navegue até ele.

## 3.7 Arquitetura em camadas

Services, repositories, adapters e helpers organizados em camadas com regras claras de quem pode importar quem. A interface não deve saber como os dados chegam.

> **NO CONNECT**
> O `app_main` separa `core`, `domain`, `modules` e `ui`.

**PARA ESTUDAR**

- Código do app_main
- Livro: Clean Architecture (Robert C. Martin), capítulos sobre dependências

**PRATIQUE**

3.7.1 Siga uma funcionalidade do botão na tela até o service que fala com o BLE e nomeie cada camada.
3.7.2 Encontre uma dependência que viole as camadas, ou mostre que não há.

## 3.8 Pacotes locais e monorepo

Dependências por `path`, submódulos git e reuso de pacotes entre projetos. Permite compartilhar código sem publicar nada.

> **NO CONNECT**
> O app usa pacotes locais como `we_bluetooth` e `we_exception`.

**PARA ESTUDAR**

- Dart: package dependencies

**PRATIQUE**

3.8.1 Altere uma função num pacote local e verifique a mudança no app sem publicar nada.
3.8.2 Explique o que acontece se o submódulo de um pacote estiver num commit diferente do esperado.

## 3.9 Serialização e geração de código

`json_serializable`, `build_runner` e `equatable`. Gerar o código repetitivo evita erros de digitação em centenas de campos.

> **NO CONNECT**
> O app usa `json_serializable`, `build_runner` e `equatable`.

**PARA ESTUDAR**

› json_serializable no pub.dev
› equatable no pub.dev

**PRATIQUE**

3.9.1 Crie um modelo com `json_serializable`, gere o código e escreva um teste de ida e volta.
3.9.2 Explique por que estados do BLoC costumam estender `Equatable`.

---

## 3.10 Internacionalização

Textos traduzidos, datas, números e formatos por idioma. Pensar em idiomas desde o início custa pouco; adaptar depois custa caro.

> **NO CONNECT**
> O app usa `slang` e `intl`.

**PARA ESTUDAR**

› slang no pub.dev
› intl no pub.dev

**PRATIQUE**

3.10.1 Adicione uma string nova em português e inglês e use-a numa tela.
3.10.2 Formate a data de uma sessão nos dois idiomas.

---

## 3.11 Persistência local

Banco local, armazenamento seguro e caminhos de arquivos. Dados de sessão e credenciais exigem cuidados diferentes.

> **NO CONNECT**
> O app usa Hive, armazenamento seguro de sessão e `path_provider`.

**PARA ESTUDAR**

› hive no pub.dev
› path_provider no pub.dev

**PRATIQUE**

3.11.1 Liste o que o app guarda localmente e onde.
3.11.2 Explique o que não deve ir para armazenamento comum e por quê.

## 3.12 Formulários e validação

Formulários, máscaras de entrada e validação de dados como CPF. Validar na entrada é mais barato que corrigir no banco.

> **NO CONNECT**
> O app tem máscaras, validação de CPF e helpers de formato.

**PARA ESTUDAR**

› Flutter: form validation

**PRATIQUE**

3.12.1 Implemente a validação de CPF pelos dígitos verificadores e escreva testes com casos válidos e inválidos.
3.12.2 Adicione uma máscara de telefone a um campo existente.

## 3.13 Lint e qualidade

`analysis_options.yaml`, regras de lint e `dart format`. O analisador estático pega erros antes de o app rodar.

> **NO CONNECT**
> O projeto usa as regras do `flutter_lints`.

**PARA ESTUDAR**

› Dart: customizing static analysis

**PRATIQUE**

3.13.1 Rode `flutter analyze` e corrija três avisos.
3.13.2 Proponha uma regra de lint a mais para o projeto e justifique.

**NÍVEL AVANÇADO** 3.14 a 3.24, 11 tópicos

*BLE, bytes, backend, fluxos clínicos, testes e publicação.*

## 3.14 Bluetooth Low Energy: lado central

Scan, conexão, descoberta de serviços, leitura, escrita e notify, permissões e diferenças entre Android e iOS. É o lado do celular na conversa BLE.

> **NO CONNECT**
> O app usa `flutter_reactive_ble`, junto com o pacote local `bluetooth`.

**PARA ESTUDAR**

- flutter_reactive_ble no pub.dev
- Nordic DevAcademy: Bluetooth Low Energy Fundamentals

**PRATIQUE**

3.14.1 Escreva um app mínimo que encontre o Connect, conecte e mostre o nível de bateria em tempo real.
3.14.2 Trate a desconexão no meio de uma sessão e defina o que a interface deve mostrar.

## 3.15 Conversão de bytes

Serialização binária com `ByteData`, endianness e o espelho, no app, das structs definidas no firmware. Um byte fora do lugar muda o valor inteiro.

> **NO CONNECT**
> O pacote `byte_conversion` faz essa ponte com o firmware.

**PARA ESTUDAR**

- ByteData (dart:typed_data)
- Pacote byte_conversion do repositório

**PRATIQUE**

3.15.1 Converta uma struct do firmware (com `int16`, `uint32` e `float`) para bytes e de volta, com teste.
3.15.2 Mostre o bug que aparece quando app e firmware discordam do endianness.

## 3.16 Contrato entre app e firmware

UUIDs, layout de cada característica, versionamento de protocolo e como evoluir sem quebrar dispositivos ou apps antigos. O contrato é o documento mais importante entre duas equipes.

> **NO CONNECT**
> O contrato do Connect está no documento técnico e no pacote `bluetooth`.

**PARA ESTUDAR**

› Documento técnico do Connect e código do pacote bluetooth

**PRATIQUE**

3.16.1 Monte uma tabela característica × layout de bytes × quem escreve × quem lê.
3.16.2 Proponha como o app deve se comportar ao encontrar um firmware com versão de protocolo mais nova.

## 3.17 Concorrência no app

Streams, locks assíncronos e controle de comandos simultâneos, para que uma escrita não atropele outra.

> **NO CONNECT**
> O app usa o pacote `synchronized` para serializar comandos BLE.

**PARA ESTUDAR**

› synchronized no pub.dev
› Dart: asynchronous programming

**PRATIQUE**

3.17.1 Dispare duas escritas BLE ao mesmo tempo sem lock e observe o resultado; depois proteja com `synchronized`.
3.17.2 Explique a diferença entre isolates e código assíncrono no mesmo isolate.

## 3.18 Comunicação com backend

HTTP (interceptors, timeouts), gRPC, autenticação JWT e upload de arquivos. É como o app guarda e busca dados fora do celular.

> **NO CONNECT**
> O app usa Dio para HTTP, gRPC e autenticação JWT.

**PARA ESTUDAR**

› dio no pub.dev
› gRPC para Dart
› Introdução a JWT

**PRATIQUE**

3.18.1 Mapeie as chamadas do app ao backend e quais usam HTTP e quais usam gRPC.
3.18.2 Explique como o app renova um token expirado sem deslogar o usuário.

## 3.19 Firebase

Autenticação, banco de documentos, notificações push e relatórios de falha como serviço.

> **NO CONNECT**
> O app usa Firebase Auth, Firestore, Cloud Messaging e Crashlytics.

**PARA ESTUDAR**

› Firebase para Flutter

**PRATIQUE**

3.19.1 Liste o que o app usa de cada serviço Firebase.
3.19.2 Force um erro não tratado num build de teste e encontre-o no Crashlytics.

## 3.20 Tratamento de erros padronizado

Códigos de erro únicos e mapeamento de falhas de rede, de protocolo e de dispositivo para mensagens que o usuário entende.

> **NO CONNECT**
> O pacote `we_exception` padroniza os erros e mapeia respostas HTTP e gRPC.

**PARA ESTUDAR**

› Código do pacote we_exception

**PRATIQUE**

3.20.1 Siga um erro 401 do backend até a mensagem exibida na tela.
3.20.2 Adicione um código de erro para falha de conexão BLE e use-o em um ponto do app.

## 3.21 Fluxos clínicos no app

Pré e pós-sessão, execução e interrupção de exercício, relatório e agenda. Em um app de saúde, o fluxo do usuário é parte da segurança.

> **NO CONNECT**
> Os módulos `fes`, `breath`, `patient` e `medic` implementam esses fluxos, incluindo o chat de pré e pós- sessão.

**PARA ESTUDAR**

› Código dos módulos fes, breath, patient e medic

**PRATIQUE**

3.21.1 Desenhe a sequência de telas de uma sessão completa, do login ao relatório.
3.21.2 Liste todos os caminhos pelos quais uma sessão pode ser interrompida e o que acontece no dispositivo em cada um.

## 3.22 Gráficos e visualização

Gráficos de sessão e de evolução, e indicadores de progresso. Bons gráficos ajudam o profissional a decidir.

> **NO CONNECT**
> O app usa Syncfusion Charts.

**PARA ESTUDAR**

› syncfusion_flutter_charts no pub.dev

**PRATIQUE**

3.22.1 Crie um gráfico de linha com a amplitude ao longo de uma sessão simulada.
3.22.2 Verifique a licença do Syncfusion usada pelo projeto e suas condições.

## 3.23 Testes

Testes unitários, de widget e de integração, e mocks da camada de hardware para testar sem dispositivo.

> **NO CONNECT**
> Com um mock do serviço BLE, a maior parte do app pode ser testada sem o Connect por perto.

**PARA ESTUDAR**

› Flutter: testing
› mocktail no pub.dev

**PRATIQUE**

3.23.1 Escreva um teste de Cubit para o fluxo iniciar → executando → parar.
3.23.2 Crie um mock do serviço BLE e teste a tela de controle sem dispositivo.

---

## 3.24 Distribuição

Builds de release, Play Store e App Store, trilhas de teste e versionamento.

> **NO CONNECT**
> A versão do app é definida no campo `version` do `pubspec.yaml`.

**PARA ESTUDAR**

› Flutter: deployment

**PRATIQUE**

3.24.1 Descreva o processo de release do app passo a passo (vira checklist da equipe).
3.24.2 Explique a diferença entre `versionName` e `versionCode` e como o pubspec define os dois.

PARTE 04

# Integração do sistema

A trilha final, comum a todas as áreas. Aqui o membro deixa de olhar só para a sua camada e passa a responder pelo sistema inteiro: do toque na tela até a corrente no eletrodo.

## VOCÊ VAI APRENDER A

- √ Seguir um comando por todas as camadas e localizar onde um problema nasce.
- √ Implementar uma funcionalidade ponta a ponta, do firmware ao app.
- √ Documentar, analisar riscos e validar um sistema completo.

## TÓPICOS DESTA PARTE


**4.1** O caminho completo de um comando

**4.2** Uma funcionalidade ponta a ponta

**4.3** Depuração entre camadas

**4.4** Documentação de engenharia

**4.5** Análise de segurança do sistema

**4.6** Verificação e validação

## 4.1 O caminho completo de um comando

Todo sistema embarcado conectado tem uma cadeia de ida (comando) e de volta (telemetria). Conhecer cada elo, sua latência e seus pontos de falha é o que separa quem conserta sintomas de quem resolve causas.

> **NO CONNECT**
> Ida: app → BLE → ESP32 → UART → ATSAM → DAC → fonte de corrente → eletrodos. Volta: bateria, impedância e estado fazem o caminho inverso.

**PARA ESTUDAR**

- Documento técnico do Connect
- Tópicos 2.21, 2.22, 2.26, 3.14 e 3.16 deste documento

**PRATIQUE**

4.1.1 Desenhe um diagrama de sequência (Mermaid) do comando “iniciar estímulo”, com todas as mensagens e camadas.
4.1.2 Estime a latência de cada trecho e meça a latência total do toque ao primeiro pulso.

## 4.2 Uma funcionalidade ponta a ponta

Adicionar uma funcionalidade a um produto com várias camadas exige mudanças coordenadas e compatíveis em cada uma. Planejar antes de codar evita retrabalho.

> **NO CONNECT**
> Estudo de caso: o GVS, que exigiu quatro passos no ATSAM, uma característica nova no ESP32 e suporte no app.

**PARA ESTUDAR**

- Histórico de commits e PRs da implementação do GVS nos três repositórios

**PRATIQUE**

4.2.1 Reconstrua, a partir dos commits, a lista de mudanças que o GVS exigiu em cada repositório.
4.2.2 Escreva o plano de implementação de uma funcionalidade simples (por exemplo, um contador de sessões) nas três camadas.

## 4.3 Depuração entre camadas

Sniffer BLE, analisador lógico e logs com horário correlacionados. A técnica é sempre a mesma: observar a fronteira entre duas camadas para descobrir de qual lado o problema está.

> **NO CONNECT**
> As fronteiras observáveis do Connect são o enlace BLE (nRF Connect) e a UART entre ESP32 e ATSAM (analisador lógico).

**PARA ESTUDAR**

- sigrok / PulseView
- Saleae Logic
- App nRF Connect for Mobile (Nordic)

**PRATIQUE**

4.3.1 Capture ao mesmo tempo o tráfego BLE e a UART durante um comando e alinhe os eventos numa linha do tempo.
4.3.2 Introduza um atraso artificial no firmware BLE e localize-o só pelas capturas.

## 4.4 Documentação de engenharia

Documento técnico e diagramas de arquitetura, sequência e estados, mantidos junto do código. Documentação boa é a que o próximo membro consegue usar sem perguntar.

> **NO CONNECT**
> Módulos novos, como o de tDCS, devem terminar com um documento técnico próprio.

**PARA ESTUDAR**

- Mermaid
- C4 model, para diagramas de arquitetura

**PRATIQUE**

4.4.1 Escreva a documentação técnica de um módulo que você estudou, com ao menos um diagrama.
4.4.2 Revise a documentação de um colega e aponte o que não daria para entender sem ler o código.

## 4.5 Análise de segurança do sistema

FMEA, árvore de falhas e o mapeamento de cada mitigação entre hardware, firmware e app. Em equipamento médico, todo risco relevante precisa de pelo menos uma barreira registrada.

> **NO CONNECT**
> Um bom exercício é perda de BLE durante o estímulo: o que o app, o ESP32, o ATSAM e o hardware fazem.

**PARA ESTUDAR**

- ISO 14971 (gestão de risco)
- Material de FMEA de disciplinas de confiabilidade ou da equipe

**PRATIQUE**

4.5.1 Faça o FMEA de uma falha com causa, efeito, severidade e mitigação em cada camada.
4.5.2 Monte a árvore de falhas do evento “corrente acima do limite no paciente”.

## 4.6 Verificação e validação

Testes de bancada, testes ponta a ponta, regressão e rastreabilidade de requisito até teste. Verificar é provar que se construiu certo; validar é provar que se construiu a coisa certa.

> **NO CONNECT**
> Um roteiro de regressão ponta a ponta antes de cada release protege as três bases de código ao mesmo tempo.

**PARA ESTUDAR**

› IEC 62304 (verificação de software médico)
› Tópicos 1.19, 2.29 e 3.23 deste documento

**PRATIQUE**

4.6.1 Monte uma matriz de rastreabilidade para três requisitos: requisito → implementação → teste → resultado.
4.6.2 Escreva um roteiro de regressão ponta a ponta que possa ser rodado antes de cada release.

APÊNDICE

# Glossário

Termos que aparecem ao longo do documento, em ordem alfabética.

| TERMO | SIGNIFICADO |
|---|---|
| **ADC / DAC** | Conversor analógico-digital e digital-analógico: traduzem tensões em números e números em tensões. |
| **BLE** | Bluetooth Low Energy, versão de baixo consumo do Bluetooth usada entre o celular e o dispositivo. |
| **BLoC / Cubit** | Padrão de gerência de estado no Flutter que separa a lógica da interface. |
| **Bootstrap** | Técnica de gate driver que usa um capacitor carregado para acionar o transistor do lado alto de uma ponte. |
| **CCC** | Client Characteristic Configuration: descritor BLE que liga ou desliga notify e indicate. |
| **Checksum** | Valor calculado sobre um quadro de dados para detectar erros de transmissão. |
| **Compliance** | Maior tensão que uma fonte de corrente consegue aplicar mantendo a corrente programada. |
| **Debounce** | Filtragem do ruído mecânico de botões e chaves. |
| **Endianness** | Ordem dos bytes de um número na memória ou num protocolo (little-endian ou big-endian). |
| **ESR** | Resistência série equivalente de um capacitor; define perdas e ripple. |
| **FES** | Estimulação elétrica funcional: pulsos que provocam contração muscular útil. |
| **FMEA** | Análise de modos de falha e efeitos: tabela de falhas possíveis, consequências e mitigações. |
| **GATT** | Estrutura de services e characteristics pela qual um periférico BLE expõe dados. |
| **GVS** | Estimulação vestibular galvânica: corrente aplicada atrás das orelhas que atua no equilíbrio. |
| **ISR** | Rotina de serviço de interrupção, código executado quando um evento de hardware acontece. |
| **Jitter** | Variação indesejada no tempo de um evento que deveria ser periódico. |
| **Latch** | Circuito que mantém um estado (por exemplo, falha) até ser rearmado. |
| **MTU** | Maior pacote de dados aceito num enlace; no BLE define quantos bytes cabem numa operação. |
| **NVIC** | Controlador de interrupções dos núcleos ARM Cortex-M. |
| **Open-drain** | Saída que só puxa para o terra; precisa de pull-up para gerar nível alto. |

| TERMO | SIGNIFICADO |
|---|---|
| **RAII** | Técnica de C++ que amarra a aquisição e a liberação de um recurso ao ciclo de vida de um objeto. |
| **SERCOM** | Periférico serial configurável do ATSAMC20 (SPI, I²C ou USART). |
| **Shunt** | Resistor de baixo valor usado para medir corrente pela queda de tensão. |
| **SWD** | Serial Wire Debug: interface de gravação e depuração dos ARM Cortex-M. |
| **tDCS** | Estimulação transcraniana por corrente contínua, de baixa intensidade, aplicada no couro cabeludo. |
| **UART** | Interface serial assíncrona de dois fios (TX e RX). |
| **Watchdog** | Temporizador que reinicia o sistema se o firmware parar de sinalizar que está vivo. |

APÊNDICE

# Ficha de progresso

Marque cada tópico concluído e anote a data. A ficha também ajuda quem acompanha sua formação a saber onde você está.

## Parte 0: Fundamentos comuns

| | Nº | TÓPICO | NÍVEL | CONCLUÍDO EM |
|---|---|---|---|---|
| ☐ | 0.1 | Dispositivos de neuroestimulação | Comum | ____/____/_____ |
| ☐ | 0.2 | Fisiologia básica para engenharia | Comum | ____/____/_____ |
| ☐ | 0.3 | Segurança elétrica e normas de equipamento médico | Comum | ____/____/_____ |
| ☐ | 0.4 | Sistemas numéricos e representação de dados | Comum | ____/____/_____ |
| ☐ | 0.5 | Linha de comando Linux | Comum | ____/____/_____ |
| ☐ | 0.6 | Git e fluxo de trabalho | Comum | ____/____/_____ |
| ☐ | 0.7 | Leitura de documentação técnica | Comum | ____/____/_____ |

## Parte 1: Hardware

| | Nº | TÓPICO | NÍVEL | CONCLUÍDO EM |
|---|---|---|---|---|
| ☐ | 1.1 | Circuitos elétricos | Básico | ____/____/_____ |
| ☐ | 1.2 | Componentes passivos | Básico | ____/____/_____ |
| ☐ | 1.3 | Semicondutores | Básico | ____/____/_____ |
| ☐ | 1.4 | Instrumentação de bancada | Básico | ____/____/_____ |

| | | | | |
|---|---|---|---|---|
| ☐ | 1.5 | Leitura de esquemático | Básico | ____/____/_____ |
| ☐ | 1.6 | Amplificadores operacionais | Intermediário | ____/____/_____ |
| ☐ | 1.7 | Fontes de corrente controladas por tensão | Intermediário | ____/____/_____ |
| ☐ | 1.8 | Medição de corrente | Intermediário | ____/____/_____ |
| ☐ | 1.9 | Lógica digital | Intermediário | ____/____/_____ |
| ☐ | 1.10 | Conversores DAC e ADC | Intermediário | ____/____/_____ |
| ☐ | 1.11 | Chaves analógicas e multiplexação | Intermediário | ____/____/_____ |
| ☐ | 1.12 | Relés de estado sólido e isolação | Intermediário | ____/____/_____ |
| ☐ | 1.13 | Fontes chaveadas | Avançado | ____/____/_____ |
| ☐ | 1.14 | Gate drivers e ponte H | Avançado | ____/____/_____ |
| ☐ | 1.15 | Gerenciamento de bateria | Avançado | ____/____/_____ |
| ☐ | 1.16 | Sequenciamento de energia e load switches | Avançado | ____/____/_____ |
| ☐ | 1.17 | Proteções de hardware | Avançado | ____/____/_____ |
| ☐ | 1.18 | Integridade de sinal e layout | Avançado | ____/____/_____ |
| ☐ | 1.19 | Validação de hardware | Avançado | ____/____/_____ |

## Parte 2: Firmware

| | Nº | TÓPICO | NÍVEL | CONCLUÍDO EM |
|---|---|---|---|---|
| ☐ | 2.1 | Linguagem C | Básico | ____/____/_____ |

| | | | | |
|---|---|---|---|---|
| ☐ | **2.2** | C++ para embarcados | Básico | ____/____/_____ |
| ☐ | **2.3** | Arquitetura de microcontroladores | Básico | ____/____/_____ |
| ☐ | **2.4** | GPIO | Básico | ____/____/_____ |
| ☐ | **2.5** | Toolchain | Básico | ____/____/_____ |
| ☐ | **2.6** | Build system | Básico | ____/____/_____ |
| ☐ | **2.7** | Cortex-M0+ | Intermediário | ____/____/_____ |
| ☐ | **2.8** | Startup e linker script | Intermediário | ____/____/_____ |
| ☐ | **2.9** | Periféricos de um microcontrolador | Intermediário | ____/____/_____ |
| ☐ | **2.10** | Geradores de código de fabricante | Intermediário | ____/____/_____ |
| ☐ | **2.11** | Protocolos seriais | Intermediário | ____/____/_____ |
| ☐ | **2.12** | Drivers de CIs externos | Intermediário | ____/____/_____ |
| ☐ | **2.13** | Gravação e depuração | Intermediário | ____/____/_____ |
| ☐ | **2.14** | Qualidade de código | Intermediário | ____/____/_____ |
| ☐ | **2.15** | Arquitetura superloop cooperativa | Avançado | ____/____/_____ |
| ☐ | **2.16** | Máquinas de estados | Avançado | ____/____/_____ |
| ☐ | **2.17** | Temporização precisa | Avançado | ____/____/_____ |
| ☐ | **2.18** | Concorrência com interrupções | Avançado | ____/____/_____ |

| | | | | |
|---|---|---|---|---|
| ☐ | 2.19 | Tratamento de erros | Avançado | ____/____/_____ |
| ☐ | 2.20 | Persistência | Avançado | ____/____/_____ |
| ☐ | 2.21 | Estado central e sincronização | Avançado | ____/____/_____ |
| ☐ | 2.22 | Protocolo binário próprio | Avançado | ____/____/_____ |
| ☐ | 2.23 | ESP32 e ESP-IDF | Avançado | ____/____/_____ |
| ☐ | 2.24 | FreeRTOS | Avançado | ____/____/_____ |
| ☐ | 2.25 | Bluetooth Low Energy: lado periférico | Avançado | ____/____/_____ |
| ☐ | 2.26 | Ponte entre BLE e UART | Avançado | ____/____/_____ |
| ☐ | 2.27 | Código compartilhado entre plataformas | Avançado | ____/____/_____ |
| ☐ | 2.28 | Firmware de segurança crítica | Avançado | ____/____/_____ |
| ☐ | 2.29 | Testes de firmware | Avançado | ____/____/_____ |

## Parte 3: Software

| | Nº | TÓPICO | NÍVEL | CONCLUÍDO EM |
|---|---|---|---|---|
| ☐ | 3.1 | Linguagem Dart | Básico | ____/____/_____ |
| ☐ | 3.2 | Fundamentos de Flutter | Básico | ____/____/_____ |
| ☐ | 3.3 | Ambiente de desenvolvimento | Básico | ____/____/_____ |
| ☐ | 3.4 | Projetos Android e iOS | Básico | ____/____/_____ |
| ☐ | 3.5 | Gerência de estado com BLoC | Intermediário | ____/____/_____ |

| | | | | |
|---|---|---|---|---|
| ☐ | **3.6** | Modularização e injeção de dependência | Intermediário | ____/____/_____ |
| ☐ | **3.7** | Arquitetura em camadas | Intermediário | ____/____/_____ |
| ☐ | **3.8** | Pacotes locais e monorepo | Intermediário | ____/____/_____ |
| ☐ | **3.9** | Serialização e geração de código | Intermediário | ____/____/_____ |
| ☐ | **3.10** | Internacionalização | Intermediário | ____/____/_____ |
| ☐ | **3.11** | Persistência local | Intermediário | ____/____/_____ |
| ☐ | **3.12** | Formulários e validação | Intermediário | ____/____/_____ |
| ☐ | **3.13** | Lint e qualidade | Intermediário | ____/____/_____ |
| ☐ | **3.14** | Bluetooth Low Energy: lado central | Avançado | ____/____/_____ |
| ☐ | **3.15** | Conversão de bytes | Avançado | ____/____/_____ |
| ☐ | **3.16** | Contrato entre app e firmware | Avançado | ____/____/_____ |
| ☐ | **3.17** | Concorrência no app | Avançado | ____/____/_____ |
| ☐ | **3.18** | Comunicação com backend | Avançado | ____/____/_____ |
| ☐ | **3.19** | Firebase | Avançado | ____/____/_____ |
| ☐ | **3.20** | Tratamento de erros padronizado | Avançado | ____/____/_____ |
| ☐ | **3.21** | Fluxos clínicos no app | Avançado | ____/____/_____ |
| ☐ | **3.22** | Gráficos e visualização | Avançado | ____/____/_____ |

| | | | | |
|---|---|---|---|---|
| ☐ | 3.23 | Testes | Avançado | ____/____/_____ |
| ☐ | 3.24 | Distribuição | Avançado | ____/____/_____ |

## Parte 4: Integração do sistema

| | Nº | TÓPICO | NÍVEL | CONCLUÍDO EM |
|---|---|---|---|---|
| ☐ | 4.1 | O caminho completo de um comando | Comum | ____/____/_____ |
| ☐ | 4.2 | Uma funcionalidade ponta a ponta | Comum | ____/____/_____ |
| ☐ | 4.3 | Depuração entre camadas | Comum | ____/____/_____ |
| ☐ | 4.4 | Documentação de engenharia | Comum | ____/____/_____ |
| ☐ | 4.5 | Análise de segurança do sistema | Comum | ____/____/_____ |
| ☐ | 4.6 | Verificação e validação | Comum | ____/____/_____ |

APÊNDICE

# Como contribuir

Este é um documento vivo. Sempre que estudar um tópico, deixe-o um pouco melhor: troque um link quebrado, acrescente um vídeo que ajudou, proponha um exercício ou corrija algo que mudou no código.

1. Faça a mudança no documento-fonte deste material, nunca só no PDF.
2. Siga o modelo de tópico abaixo, mantendo a numeração da trilha.
3. Confira se o quadro No Connect ainda corresponde ao código atual.
4. Registre a mudança no histórico de revisões (página “Sobre este documento”) e peça a revisão de um colega.

## Modelo de tópico

### X.Y Nome do tópico

Texto de abertura: o conceito geral, em 2 a 3 frases.

> **NO CONNECT**
> Onde o conceito aparece nos repositórios ou esquemáticos.

**PARA ESTUDAR**

› Link, livro ou vídeo (com capítulo, se fizer sentido)
› Documento ou código interno relacionado

**PRATIQUE**

4.6.1 Exercício conceitual
4.6.2 Exercício prático ligado ao código ou ao hardware real

NeuroDynamics®



# NeuroDynamics

Documento de formação técnica da equipe NeuroDynamics. Encontrou um erro ou tem uma sugestão? Veja “Como contribuir” e ajude com o desenvolvimento do documento.

